advanced Terraform concepts you should know after the basics like init, plan, apply, variables, and simple resources.

1. Meta-arguments

These control how resources are created and managed.

for_each

Used when you want to create multiple resources from a map or set, with stable keys. HashiCorp recommends it for managing several similar objects. (HashiCorp Developer)

```
resource "azurerm_resource_group" "rg" {
  for_each = {
    dev  = "East US"
    prod = "West Europe"
  }

  name     = "rg-${each.key}"
  location = each.value
}
```

count

Good for simple repeated resources, but less flexible than for_each.

depends_on

For explicit dependency when Terraform cannot infer it.

```
resource "azurerm_virtual_machine" "vm" {
  # ...
  depends_on = [azurerm_network_interface.nic]
}
```

2. Lifecycle rules

The lifecycle block customizes resource behavior during create, update, and destroy operations. (HashiCorp Developer)

Common rules:

- create_before_destroy
- prevent_destroy
- ignore_changes
- replace_triggered_by

```
resource "azurerm_storage_account" "sa" {
  name                     = "examplestorage001"
  resource_group_name      = azurerm_resource_group.rg.name
  location                 = azurerm_resource_group.rg.location
  account_tier             = "Standard"
  account_replication_type = "LRS"

  lifecycle {
    prevent_destroy = true
    ignore_changes  = [tags]
  }
```

```
}
```
This is very useful in production to avoid accidental deletion.

3. Dynamic blocks

Dynamic blocks help generate nested blocks programmatically. They are useful when a resource has repeated inner configuration. Terraform notes that dynamic blocks can generate only normal nested arguments, not meta-arguments like lifecycle. (HashiCorp Developer)

```
resource "aws_security_group" "sg" {
  name = "web-sg"

  dynamic "ingress" {
    for_each = [80, 443]
    content {
      from_port   = ingress.value
      to_port     = ingress.value
      protocol    = "tcp"
      cidr_blocks = ["0.0.0.0/0"]
    }
  }
}
```

4. Modules and reusable architecture

Advanced Terraform relies heavily on **modules** for reusable infrastructure.

Typical structure:

- modules/network
- modules/compute
- modules/database

Benefits:

- reusable code
- standardization
- easier team collaboration
- environment separation

Example:

```
module "network" {
  source              = "./modules/network"
  vnet_name           = "main-vnet"
  resource_group_name = azurerm_resource_group.rg.name
  location            = azurerm_resource_group.rg.location
}
```

Advanced module practices:

- input validation
- outputs
- version pinning
- provider alias support
- avoiding hardcoded values

5. Remote state and backends

A backend defines where Terraform stores state. Terraform docs describe backends as the mechanism for storing state and running operations. (HashiCorp Developer)

Why remote state matters:

- collaboration
- state locking
- secure shared state
- CI/CD integration

Examples of remote backends:

- AWS S3 + DynamoDB lock
- Azure Storage Account
- GCS
- HCP Terraform

Example:

```
terraform {
  backend "azurerm" {
    resource_group_name  = "tf-rg"
    storage_account_name = "tfstateacc"
    container_name       = "tfstate"
    key                  = "prod.terraform.tfstate"
  }
}
```

6. Workspaces

Workspaces let one configuration use multiple separate state files. Terraform docs explain that each configuration has a backend and workspaces keep separate state data within it. (HashiCorp Developer)

Use cases:

- dev / test / prod
- feature environments
- isolated deployments

```
terraform workspace new dev
terraform workspace new prod
terraform workspace select dev
```

Important real-world note: for large enterprise setups, separate folders or separate backends are often cleaner than overusing workspaces.

7. State management

Terraform state is one of the most advanced and important topics.

You should know:

- terraform.tfstate
- state locking
- terraform state list
- terraform state show
- terraform state mv

- terraform state rm
- terraform import

Why it matters:

- fixing drift
- refactoring resources
- moving resources into modules
- importing existing cloud resources safely

HashiCorp provides import workflows for bringing existing infrastructure under Terraform management. (HashiCorp Developer)

8. Preconditions, postconditions, and checks

Terraform supports validation inside configuration. HashiCorp recommends:

- **preconditions** to verify assumptions before creation
- **postconditions** to verify guarantees after operations (HashiCorp Developer)

Example:

```
resource "aws_instance" "app" {
  ami           = "ami-123456"
  instance_type = "t3.micro"

  lifecycle {
    precondition {
      condition     = var.environment != ""
      error_message = "Environment must not be empty."
    }
  }
}
```

This is powerful for production-grade modules.

9. Provider aliasing

Useful when deploying to multiple regions or multiple accounts/subscriptions.

```
provider "aws" {
  region = "us-east-1"
}

provider "aws" {
  alias  = "west"
  region = "us-west-2"
}

resource "aws_s3_bucket" "east_bucket" {
  bucket = "east-bucket-demo"
}

resource "aws_s3_bucket" "west_bucket" {
  provider = aws.west
  bucket   = "west-bucket-demo"
```

```
}
```
This is common in multi-region and multi-account designs.

10. Data sources

Data sources let Terraform read existing infrastructure instead of creating it.

```
data "aws_ami" "latest" {
  most_recent = true
  owners      = ["amazon"]
}
```
Advanced use:

- fetch existing VPCs/subnets
- reference secrets
- integrate with already-created infrastructure

11. Expressions and advanced functions

Terraform becomes much stronger when you use:

- for expressions
- conditional expressions
- try()
- lookup()
- merge()
- flatten()
- toset(), tomap()
- jsonencode()

Example:

```
locals {
  common_tags = merge(
    var.default_tags,
    { environment = var.environment }
  )
}
```

12. Local values

Locals help avoid repeating logic.

```
locals {
  naming_prefix = "${var.project}-${var.environment}"
}
```
Very useful for:

- naming conventions
- tag generation
- reused computed values

13. Variable validation

Advanced Terraform modules validate input early.

```
variable "instance_count" {
  type = number
```

```
  validation {
    condition     = var.instance_count > 0
    error_message = "instance_count must be greater than 0."
  }
}
```

This improves reliability.

14. Provisioners

Provisioners exist, but they should be used carefully. They are usually a last resort, not the primary design pattern.

Examples:

- local-exec
- remote-exec

Use them only when unavoidable, because they reduce predictability and portability.

15. Drift detection

Drift means actual cloud resources changed outside Terraform.

How to detect:

terraform plan

How to manage:

- import unmanaged resources
- update configuration
- avoid manual cloud-console changes

16. Refactoring without recreation

Advanced Terraform users must know how to reorganize code safely.

Typical actions:

- move resources into modules
- rename resources
- split monolithic configs

State commands and moved/import workflows are important here. This prevents unnecessary destroy/create operations. (HashiCorp Developer)

17. CI/CD integration

In real projects, Terraform is usually run through pipelines:

- GitHub Actions
- Azure DevOps
- GitLab CI
- Jenkins
- HCP Terraform

Best practices:

- run fmt, validate, and plan
- require plan review before apply
- use remote state
- store secrets securely

- separate roles for plan/apply

18. Policy and governance

In advanced enterprise environments, Terraform is combined with:

- Sentinel / policy-as-code
- OPA
- security scanning
- tagging standards
- cost controls
- RBAC and approval workflows

This is where Terraform moves from scripting to platform engineering.

19. Multi-environment architecture patterns

Common advanced patterns:

- separate folders per environment
- reusable modules
- remote backend per environment
- different variable files
- environment-specific pipelines

Example structure:

```
terraform/
  modules/
    network/
    compute/
  env/
    dev/
    test/
    prod/
```

This is usually better than putting everything in one file.

20. Testing and validation

Advanced teams also add:

- terraform validate
- terraform fmt
- static analysis
- module tests
- security checks

This helps prevent broken infrastructure before deployment.

Sequence:

1. modules
2. for_each and dynamic blocks
3. remote state and backends
4. state commands and import
5. lifecycle rules
6. workspaces

7. provider aliasing
8. validations, preconditions, postconditions
9. CI/CD and governance